

Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Інститут інформаційно-комунікаційних технологій та електронної інженерії
Кафедра «Інформаційно-комунікаційних технологій»



Звіт з лабораторної роботи №2
з дисципліни «Об'єктно-орієнтоване програмування, частина 2»

Підготував:
ст. групи IX-21
Петриченко Сергій

Прийняла:
док. філософії, доцент
Гордійчук-Бублівська О.В.

Львів 2026 р.

Тема роботи: Моделювання телекомунікаційної мережі з використанням генераторів подій, протоколів TCP/UDP, асинхронного моделювання. Розробка графічної візуалізації мережі та аналіз продуктивності.

Мета роботи: Ознайомлення з основами моделювання телекомунікаційних мереж. Вивчення використання об'єктно-орієнтованого програмування для моделювання мережі. Моделювання різних топологій (зірка, кільцева, деревоподібна, сіткова (mesh), комбінована тощо). Симуляція передачі пакетів між вузлами та маршрутизаторами з використанням TCP та UDP. Вимірювання продуктивності мережі: швидкість передачі, статистика втрат, затримки передачі. Побудова візуалізації для графічного відображення структури та роботи мережі.

Теоретичні відомості

Об'єктно-орієнтоване моделювання мереж. Використання ООП у Python дозволяє створювати програмні моделі реальних мережевих компонентів:

- Вузли (Node) та Маршрутизатори (Router): представляють кінцеві пристрої або активне мережеве обладнання.
- Пакети (Packet): містять дані про відправника, отримувача, розмір та тип протоколу.
- З'єднання (Edges): ребра графа, що визначають фізичні або логічні зв'язки між вузлами.

Мережеві протоколи: TCP та UDP. Для моделювання передачі даних використовуються два основні протоколи транспортного рівня:

- TCP (Transmission Control Protocol): надійний з'єднувальний протокол. Він забезпечує гарантовану доставку пакетів, контроль помилок та підтвердження отримання даних.
- UDP (User Datagram Protocol): швидкий безз'єднувальний протокол. Він не гарантує доставку або порядок пакетів, що робить його простішим, але менш надійним порівняно з TCP.

Асинхронність та генератори подій. Для імітації роботи мережі в реальному часу застосовуються механізми асинхронності:

- Бібліотека `asyncio`: дозволяє обробляти мережеві події (передачу пакетів) без блокування основного потоку програми.
- Генератори подій: механізми, що створюють потік подій, на які програма реагує по мірі їх виникнення.
- Патерн Observer: спостерігачі підписуються на події та отримують сповіщення про їх настання.

Топології мережі та візуалізація. Топологія визначає структуру зв'язків у мережі. Для їх моделювання використовується бібліотека `networkx`, а для відображення `matplotlib`.

Основні типи топологій:

- Зіркова (Star): усі вузли підключені до центрального маршрутизатора.

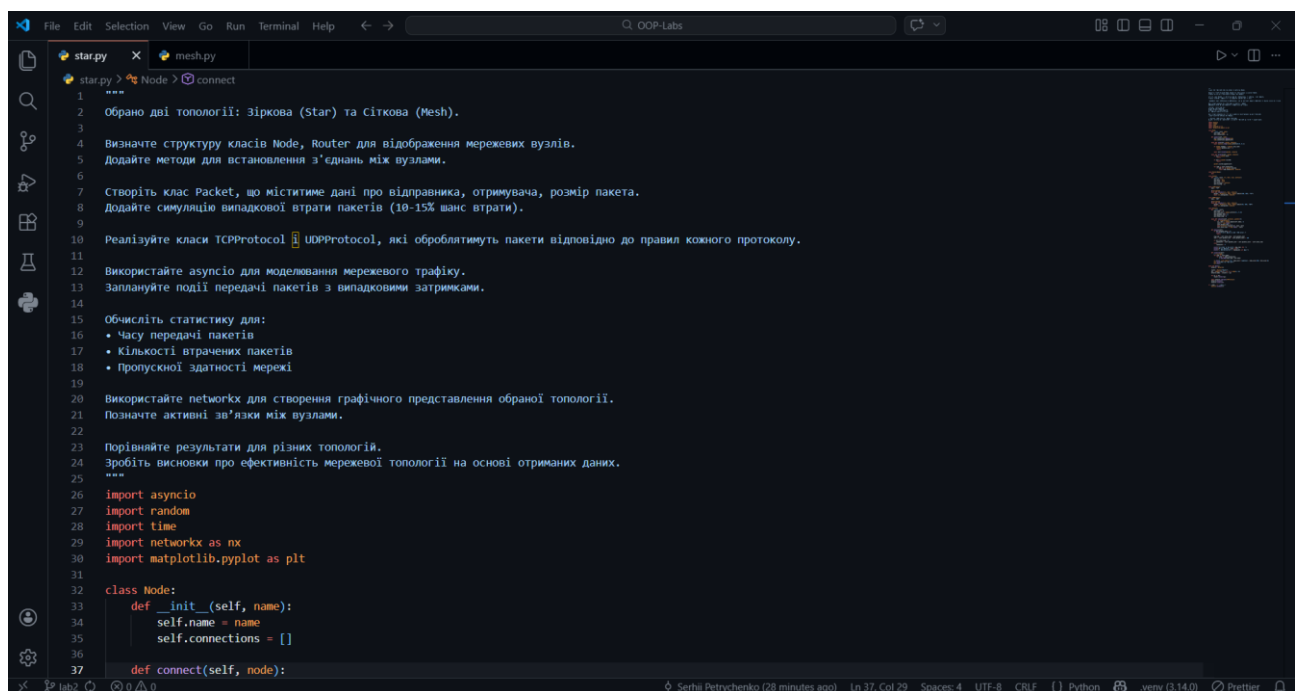
- Сіткова (Mesh) та Повнозв'язка: вузли мають численні або прямі з'єднання один з одним.
- Кільцева (Ring), Шинна (Bus) та Деревоподібна (Tree).

Метрики продуктивності. Під час симуляції вимірюються ключові показники ефективності мережі:

- Середній час передачі: затримка між відправленням та отриманням.
- Статистика втрат: відсоток пакетів, що не досягли цілі (у симуляції зазвичай 10-15%).
- Пропускна здатність: кількість успішно переданих пакетів за одиницю часу.

Результати виконаної роботи

Зіркова (Star) топологія.



```

1  ****
2  Обрано дві топології: Зіркова (Star) та Сіткова (Mesh).
3
4  Визначте структуру класів Node, Router для відображення мережевих вузлів.
5  Додайте методи для встановлення з'єднань між вузлами.
6
7  Створіть клас Packet, що міститиме дані про відправника, отримувача, розмір пакета.
8  Додайте симуляцію випадкової втрати пакетів (10-15% шанс втрати).
9
10 Реалізуйте класи TCPProtocol та UDPProtocol, які оброблятимуть пакети відповідно до правил кожного протоколу.
11
12 Використайте asyncio для моделювання мережевого трафіку.
13 Заплануйте події передачі пакетів з випадковими затримками.
14
15 Обчисліть статистику для:
16 • Часу передачі пакетів
17 • Кількості втрачених пакетів
18 • Пропускної здатності мережі
19
20 Використайте networkx для створення графічного представлення обраної топології.
21 Позначте активні зв'язки між вузлами.
22
23 Порівняйте результати для різних топологій.
24 Зробіть висновки про ефективність мережевої топології на основі отриманих даних.
25 ****
26 import asyncio
27 import random
28 import time
29 import networkx as nx
30 import matplotlib.pyplot as plt
31
32 class Node:
33     def __init__(self, name):
34         self.name = name
35         self.connections = []
36
37     def connect(self, node):

```

```
File Edit Selection View Go Run Terminal Help < - -> Q. OOP-Labs
star.py x mesh.py
star.py > Packet > __init__
32 class Node:
33
34
35
36
37     def connect(self, node):
38         self.connections.append(node)
39         node.connections.append(self)
40
41     async def send(self, packet, network):
42         await asyncio.sleep(random.uniform(0.05, 0.3))
43
44         if random.random() < network.loss_rate:
45             network.packets_lost += 1
46             return
47
48         await self.forward(packet, network)
49
50     async def forward(self, packet, network):
51         if self == packet.dest:
52             return
53
54         if self in packet.visited:
55             return
56
57         packet.visited.append(self)
58
59         for node in self.connections:
60             if node not in packet.visited:
61                 await node.send(packet, network)
62
63 class Router(Node):
64     pass
65
66 class Packet:
67     def __init__(self, src, dest, size, protocol):
68         self.src = src
69         self.dest = dest
70         self.size = size
71         self.protocol = protocol
```

```
File Edit Selection View Go Run Terminal Help < - -> Q. OOP-Labs
star.py x mesh.py
star.py > Network > analyze
72     self.visited = []
73
74 class TCPProtocol:
75     name = "TCP"
76
77     @staticmethod
78     async def transmit(src, dest, network):
79         packet = Packet(src, dest, random.randint(200, 500), "TCP")
80         await src.send(packet, network)
81
82 class UDPProtocol:
83     name = "UDP"
84
85     @staticmethod
86     async def transmit(src, dest, network):
87         packet = Packet(src, dest, random.randint(50, 200), "UDP")
88         await src.send(packet, network)
89
90 class Network:
91     def __init__(self):
92         self.nodes = []
93         self.loss_rate = random.uniform(0.1, 0.15)
94         self.packets_sent = 0
95         self.packets_lost = 0
96         self.total_time = 0
97
98     async def simulate(self, protocol, packets=5):
99         for _ in range(packets):
100             src, dest = random.sample(self.nodes, 2)
101             start = time.time()
102             self.packets_sent += 1
103             await protocol.transmit(src, dest, self)
104             self.total_time += time.time() - start
105
106     def analyze(self):
107         if self.packets_sent == 0:
108             print("Жодного пакета не було відправлено.")
```

```
File Edit Selection View Go Run Terminal Help Q: OOP-Labs
star.py x mesh.py
star.py > main
90 class Network:
105
106 def analyze(self):
107     if self.packets_sent == 0:
108         print("Жодного пакета не було відправлено.")
109         return
110
111     avg_time = self.total_time / self.packets_sent
112     loss = (self.packets_lost / self.packets_sent) * 100
113
114     if self.total_time > 0:
115         bandwidth = (self.packets_sent - self.packets_lost) / self.total_time
116     else:
117         bandwidth = 0
118
119     print(f"Середній час передачі: {avg_time:.4f} c")
120     print(f"Втрати пакетів: {loss:.2f}%")
121     print(f"Пропускна здатність: {bandwidth:.2f} пак/c")
122
123 def visualize(self):
124     G = nx.Graph()
125     for node in self.nodes:
126         for conn in node.connections:
127             G.add_edge(node.name, conn.name)
128
129     nx.draw(G, with_labels=True, node_color='lightblue', node_size=2500, font_size=10)
130     plt.title("Зіркова топологія")
131     plt.show()
132
133 async def main():
134     network = Network()
135
136     router = Router("Router")
137     pcs = [Node(f"PC{i}") for i in range(1, 5)]
138     network.nodes = [router] + pcs
139
140     for pc in pcs:
```

```
File Edit Selection View Go Run Terminal Help Q: OOP-Labs
star.py x mesh.py
star.py > ...
90 class Network:
123 def visualize(self):
124     nx.draw(G, with_labels=True, node_color='lightblue', node_size=2500, font_size=10)
125     plt.title("Зіркова топологія")
126     plt.show()
127
128 async def main():
129     network = Network()
130
131     router = Router("Router")
132     pcs = [Node(f"PC{i}") for i in range(1, 5)]
133     network.nodes = [router] + pcs
134
135     for pc in pcs:
136         router.connect(pc)
137
138     await network.simulate(TCPProtocol)
139     network.analyze()
140     network.visualize()
141
142 if __name__ == "__main__":
143     asyncio.run(main())
144
145
146
147
148
```

Figure 1

(x, y) = (0.225, 0.395)

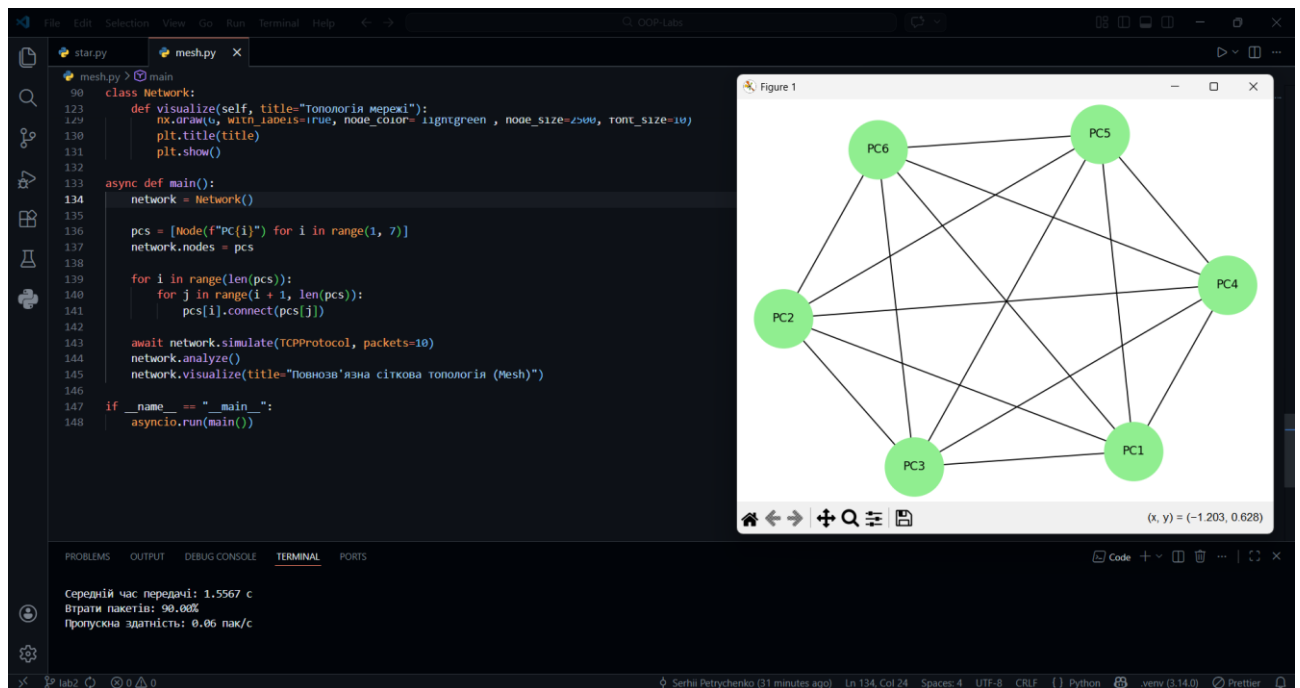
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Середній час передачі: 0.4037 c
Втрати пакетів: 60.00%
Пропускна здатність: 0.99 пак/c

Сіткова (Mesh) топологія.

```
File Edit Selection View Go Run Terminal Help Q. OOP-Labs
mesh.py mesh.py X
mesh.py > Node > connect
1 """
2 Обрано дві топології: Зіркова (Star) та Сіткова (Mesh).
3
4 Визначте структуру класів Node, Router для відображення мережеских вузлів.
5 Додайте методи для встановлення з'єднань між вузлами.
6
7 Створіть клас Packet, що міститиме дані про відправника, отримувача, розмір пакета.
8 Додайте симуляцію випадкової втрати пакетів (10-15% шанс втрати).
9
10 Реалізуйте класи TCPProtocol, UDPProtocol, які оброблятимуть пакети відповідно до правил кожного протоколу.
11
12 Використайте asyncio для моделювання мережевого трафіку.
13 Заплануйте події передачі пакетів з випадковими затримками.
14
15 Обчисліть статистику для:
16 • Часу передачі пакетів
17 • Кількості втрачених пакетів
18 • Пропускної здатності мережі
19
20 Використайте networkx для створення графічного представлення обраної топології.
21 Позначте активні зв'язки між вузлами.
22
23 Порівняйте результати для різних топологій.
24 Зробіть висновки про ефективність мережевої топології на основі отриманих даних.
25 """
26 import asyncio
27 import random
28 import time
29 import networkx as nx
30 import matplotlib.pyplot as plt
31
32 class Node:
33     def __init__(self, name):
34         self.name = name
35         self.connections = []
36
37     def connect(self, node):
```

```
File Edit Selection View Go Run Terminal Help Q. OOP-Labs
mesh.py mesh.py X
mesh.py > Packet > __init__
32 class Node:
33
34     def connect(self, node):
35         self.connections.append(node)
36         node.connections.append(self)
37
38     async def send(self, packet, network):
39         await asyncio.sleep(random.uniform(0.05, 0.3))
40
41         if random.random() < network.loss_rate:
42             network.packets_lost += 1
43             return
44
45         await self.forward(packet, network)
46
47     async def forward(self, packet, network):
48         if self == packet.dest:
49             return
50
51         if self in packet.visited:
52             return
53
54         packet.visited.append(self)
55
56         for node in self.connections:
57             if node not in packet.visited:
58                 await node.send(packet, network)
59
60 class Router(Node):
61     pass
62
63 class Packet:
64     def __init__(self, src, dest, size, protocol):
65         self.src = src
66         self.dest = dest
67         self.size = size
68         self.protocol = protocol
```

Висновки

1. Під час лабораторної роботи було виконано моделювання телекомунікаційної мережі з використанням об'єктно-орієнтованого підходу. Зокрема, було створено класи для вузлів, маршрутизаторів та пакетів, реалізовано асинхронний механізм передачі даних за допомогою протоколів TCP і UDP, а також побудовано графічну візуалізацію різних топологій мережі.
2. Цю модель можна застосовувати для попереднього аналізу продуктивності мережевих архітектур перед їх реальним впровадженням. Вона дозволяє оцінювати пропускну здатність, тестувати стійкість мережі до втрат пакетів та вивчати поведінку трафіку в умовах випадкових затримок.
3. Робота дозволила практично опанувати навички асинхронного програмування за допомогою модуля `asyncio` для моделювання подій у реальному часі. Крім того, було отримано досвід роботи з бібліотеками `networkx` та `matplotlib` для візуалізації складних структур у вигляді графів та поглиблено розуміння різниці між надійним TCP та швидким UDP протоколами.
4. Головною перевагою такої моделі є її гнучкість, оскільки вона дозволяє легко змінювати топологію (наприклад, з зіркової на сіткову) та параметри мережі. Основним недоліком є певна ідеалізація умов, адже в симуляції використовуються спрощені механізми випадкових втрат (10, 15%) та фіксовані діапазони затримок, які не завжди враховують усі фактори реального фізичного середовища.